

Renato Dei Marini

Junior Software Engineer — Backend / DevOps / Systems

Florence / Padua, Italy — open to relocation | renato.deimarini@gmail.com

GitHub: github.com/redei-ma | LinkedIn: linkedin.com/in/renato-dei-marini

PROFILE

Developer trained at the Common Core of **42 Firenze: twelve published projects**, from low-level systems in C and C++ to containerised microservices and Kubernetes. Used to designing from scratch under hard constraints and to verifying what I write instead of taking it on trust. Looking for a first junior position in backend, DevOps or system tooling.

TECHNICAL SKILLS

- **Programming:** C, C++, TypeScript (Node.js), Bash; YAML for Kubernetes manifests, Docker Compose and configuration.
- **Tools:** Git (Git Flow branching model), Make/Makefile, GCC and Clang, valgrind; Docker and Docker Compose (multi-stage Dockerfiles), NGINX, Kubernetes (K3s, k3d, kubectl), Helm, Argo CD, Traefik, Vagrant and VirtualBox; NestJS, Prisma, PostgreSQL, Redis, MariaDB, Swagger, openssl; Ollama and LLM APIs; POSIX threads, POSIX semaphores, BSD sockets, GNU readline, MiniLibX (X11).
- **Backend and infrastructure:** microservices with per-service data ownership, REST APIs and atomic transactions, relational modelling, ORM and versioned migrations; reverse proxying with TLS, rate limiting and WebSocket upgrades; containerisation (Docker networks and volumes), infrastructure as code, GitOps.
- **Systems:** concurrency (threads, mutexes, semaphores, IPC), I/O multiplexing through one `poll()`, UNIX system programming (processes, pipes, descriptors, signals), interpreters (lexing, parsing, execution), manual memory management without leaks.

PROJECTS

ft_transcendence

github.com/redei-ma/ft_transcendence

Team project (5 people), role: Project Manager — User Service & Database. Stack: TypeScript, NestJS, Prisma, PostgreSQL, Redis, Docker and Docker Compose, NGINX

- **Platform and architecture:** an online 1v1 browser game with real-time matches, ELO matchmaking, profiles, friendships and match history. Microservice architecture: four NestJS applications (auth, users, game engine, matchmaking), PostgreSQL and Redis; ten containers on a private network behind a single NGINX gateway that terminates TLS and is the only one publishing a port.
- **The entire user-service:** 8 controllers, 9 services, 48 REST endpoints, notifications over SSE; normalised Prisma schema (11 models, 14 versioned migrations) in which each service owns its data. End-of-match writes — ELO, match record, participants and statistics — go in a single transaction, so the database is never left half-written between two services.
- **Docker infrastructure:** six multi-stage Dockerfiles, `docker-compose.yml` with healthchecks and ordered dependencies, a dev override with hot reload, NGINX (shared) with TLS, rate limiting and WebSocket upgrades; every connection over HTTPS/WSS, and the whole system starts from a clean machine with a single command.
- **Hardest problem:** on restart Docker assigns a container a new IP and NGINX kept serving the cached address (502). Solved with Docker's embedded resolver (`resolver 127.0.0.11 valid=10s`) and upstreams in variables.
- **Dependency security audit:** removed a library with known vulnerabilities and upgraded the others; remaining vulnerabilities justified in writing, separating those of the build toolchain from those that actually ship inside the container. Team coordination on a Git Flow branching model (`main/dev` plus feature branches) and four technical documents in `docs/`.

Inception-of-Things

github.com/redei-ma/Inception-of-Things

Solo project. Stack: Vagrant, VirtualBox, K3s, k3d, Docker, kubectl, Helm, Argo CD, Traefik; provisioning in Bash, configuration in YAML

- **Four Kubernetes environments** of increasing complexity: a two-node K3s cluster; three apps behind an Ingress routing on the Host header; k3d with Argo CD deploying from the manifests in the repository; a bonus with GitLab inside the cluster, closing the GitOps loop locally.
- **Hardest problem (multi-layer Helm / Kubernetes / Rails debugging):** in the automated provisioning of a self-hosted in-cluster GitLab, bootstrapping the Git project kept failing: import from a remote URL is disabled by default (not among the `import_sources`) and the background port-forward opened a race condition. Solved by replacing the REST API chain with a `gitlab-rails runner` in the toolbox pod (enables the import, creates the project, polls the status and fails explicitly); plus a 422 on login caused by Secure cookies over HTTP.
- **GitOps and portability:** `prune` and `selfHeal` reconciliation, everything inside VMs and components pinned to explicit versions; cross-architecture emulation (amd64 image on an arm64 VM via `qemu-user-static`).

minishell

github.com/redei-ma/minishell

Pair project. Stack: C, Makefile, GNU readline; `fork`, `execve`, `pipe`, `dup2`, `wait`, `signal/sigaction`, terminal handling; no other external library

- **A Bash-like shell:** it reads a line, turns it into a pipeline and runs it with quoting, variable and exit-status expansion, here-documents, redirections and seven builtins. A ten-stage architecture (validation, quote-aware

tokenisation, expansion, execution with one child process per stage).

- **Hardest problem:** the order of the phases is not free (quotes decide what counts as a metacharacter; expansion comes after splitting but before quote removal). Solved by keeping quote state in two flags the scanner consults. Signals handled in three different contexts, under the 42 Norm and the constraint of a single global variable, which may hold only the signal number.

ft_irc

github.com/redei-ma/ft_irc

Team project (3 people). Stack: C++98, Makefile, BSD sockets (*poll*, non-blocking *fcntl*, *recv/send*), STL

- **An IRC server that talks to real clients:** five classes with separated responsibilities; sixteen commands, the five required channel modes, a bot as a bonus. I/O multiplexing through a **single poll()** for reading, writing and listening, in one process on non-blocking I/O: **fork** forbidden, no external libraries, and the server must never crash.
- **Hardest problem:** TCP carries a stream, not messages; one **recv** can return half a command, or three and a half. Solved with a per-client buffer that only executes commands terminated by **\r\n**.

inception

github.com/redei-ma/inception

Solo project. Stack: Docker, Docker Compose, Debian, NGINX, PHP-FPM, WordPress, MariaDB, Redis, vsftpd, Adminer, Portainer; Bash, openssl

- **Eight services in eight containers** on a private network, one service per container and every Dockerfile hand-written (ready-made images forbidden): NGINX terminates TLS 1.2/1.3 on 443 and is the only one publishing a port, the others are reachable only from inside. Idempotent containers that initialise themselves on first start.
- **Hardest problem:** a mismatch between the volume path created by the Makefile (**\$(HOME)**) and the one mounted by compose (literal); MariaDB silently reinitialised the database on every start. Solved with a single shared **DATA_PATH** variable. Reasoning about the exposure perimeter.

OTHER PROJECTS

email-triage

github.com/redei-ma/email-triage

Personal project: customer email triage with keyword rules, a local LLM and a cloud one, measured on 25 held-out test emails. Regex for the order number and the local model for the category: 25/25 in all three runs, about half a second per email, at no cost. (*TypeScript*, *Node.js*, *Ollama*, *Gemini API*, *Docker*)

philosophers

github.com/redei-ma/philosophers

The dining philosophers in two implementations (threads and processes); race conditions, deadlock and starvation, fine-grained mutexes, clean termination. (*C*, *pthread*s, *POSIX mutexes and semaphores*)

push_swap

github.com/redei-ma/push_swap

Sorts a stack by printing the shortest sequence of operations; correctness verified on every permutation up to 8 elements (46,233 cases, no failures) with an independent simulator. (*C*, *greedy algorithm under a cost constraint*)

pipex

github.com/redei-ma/pipex

Reproduces the shell pipeline (**< in cmd1 | cmd2 > out**) with one process per command; IPC through anonymous pipes and descriptor lifetimes that never block. (*C*, *fork/pipe/dup2/execve*)

libft

github.com/redei-ma/libft

A 65-function C library (including **ft_printf** and **get_next_line**) with a 109-test suite (**make test** returns 0/1); found an ABI defect on arm64. (*C without the standard library*, *Makefile with ar*)

cpp-modules

github.com/redei-ma/cpp-modules

The ten C++ modules of the 42 curriculum (38 programs): canonical form, virtual inheritance, templates, exceptions, up to a merge-insertion sort on STL containers. (*C++98*, *STL*)

cub3D

github.com/redei-ma/cub3D

A Wolfenstein 3D-style raycasting engine (DDA, wall-strip drawing, textures, minimap, movement and collisions). Pair project: I worked mainly on the rendering. (*C*, *MiniLibX (X11)*)

EDUCATION

42 Firenze — Post-Common Core specialisation, towards the RNCP qualification 2026 – present

42 Firenze — Intensive software engineering programme 2024 – 2026

- Common Core completed. Peer-to-peer method based on hands-on projects and self-directed reading of technical documentation, with strict code reviews.

Istituto Tecnico Commerciale Statale Einaudi-Gramsci, Padua 2012 – 2017

Technical secondary school diploma in Business Administration, Finance and Marketing

Languages — Italian native; English: daily technical reading and writing.

WORK EXPERIENCE

• **Electrician** — Residential and industrial installations; teamwork and safety regulations. 2021 – 2024

• **Waiter** — High-pressure customer service, multitasking. 2018 – 2021